

Java™

Last lecture reminder



We learned about:

- Java continue keyword
- Java break keyword
- What are Java methods
- How to write and call a Java method
- How to add parameters to a Java method
- How to return a value from a Java method
- How to apply logic into Java method (like if - else statements, loops, exc…)

Method Overloading

As we learned, when creating a new Java method we need to provide its name, the return value type and the parameters this method should receive.
All the above together can be called the **method signature.**

Java provide us a way to create multiple methods with the same name but with a different signature. This process is been called **method overloading**.

Note: Multiple methods can have the same name as long as the number and/or type of parameters are different (different method signature).

Method Overloading Example

```java
public class FirstProject {
    static int plusMethod(int x, int y) {
        return x + y;
    }

    static double plusMethod(double x, double y) {
        return x + y;
    }

    public static void main(String[] args) {
        int myNum1 = plusMethod( x: 8,  y: 5);
        double myNum2 = plusMethod( x: 4.3,  y: 6.26);
        System.out.println("int: " + myNum1);
        System.out.println("double: " + myNum2);
    }
}
```

Both methods has the same name but with different signature.

```
int: 13
double: 10.559999999999999
```

Method Scope

Unlike Javascript function scope, Java makes no difference between **global scope** and **local scope.**

- **Global scope -** the main thread scope.
- **Local scope -** the method scope.

In Java any variable that will be created inside a specific scope could be accessed and modified only within this specific scope.

Meaning that variables that were created in the global scope (the main thread) could not be accessed or modified inside a local scope (Java method scope).

Java Recursion

Recursion is the technique of making a function call itself. This technique provides a way to break complicated problems down into simple problems which are easier to solve.

In order to understand how recursion is working, we first need to understand how Java methods are working.
Java method will return a value only when this value has been evaluated, meaning that until this value is clear the method won't return anything and won't be closed.

When using the recursion technique, we need to make sure that we are changing the input of the recursion method for each time we are calling it.

Java Recursion - Stop Condition

Recursion stop condition - Just as loops can run into the problem of infinite looping, recursive functions can run into the problem of infinite recursion.

Infinite recursion is when the function never stops calling itself. Every recursive function should have a stop condition, which is the condition where the function stops calling itself.

```
public class FirstProject {
    public static int sum(int start, int end) {
        if (end > start) {
            return end + sum(start, end - 1);
        } else {
            return end;
        }
    }

    public static void main(String[] args) {
        int result = sum(5, 10);
        System.out.println(result);
    }
}
```

The stop condition is when end <= start, then our recursion method stop calling itself and return the end variable value

45

Java Recursion Another Example

Let's see a simple Java recursion example → **factorial method**

The **factorial method** takes 1 number as an input and return it's factorial.

For example: for the input 4 the factorial method will return 24 (4*3*2*1)

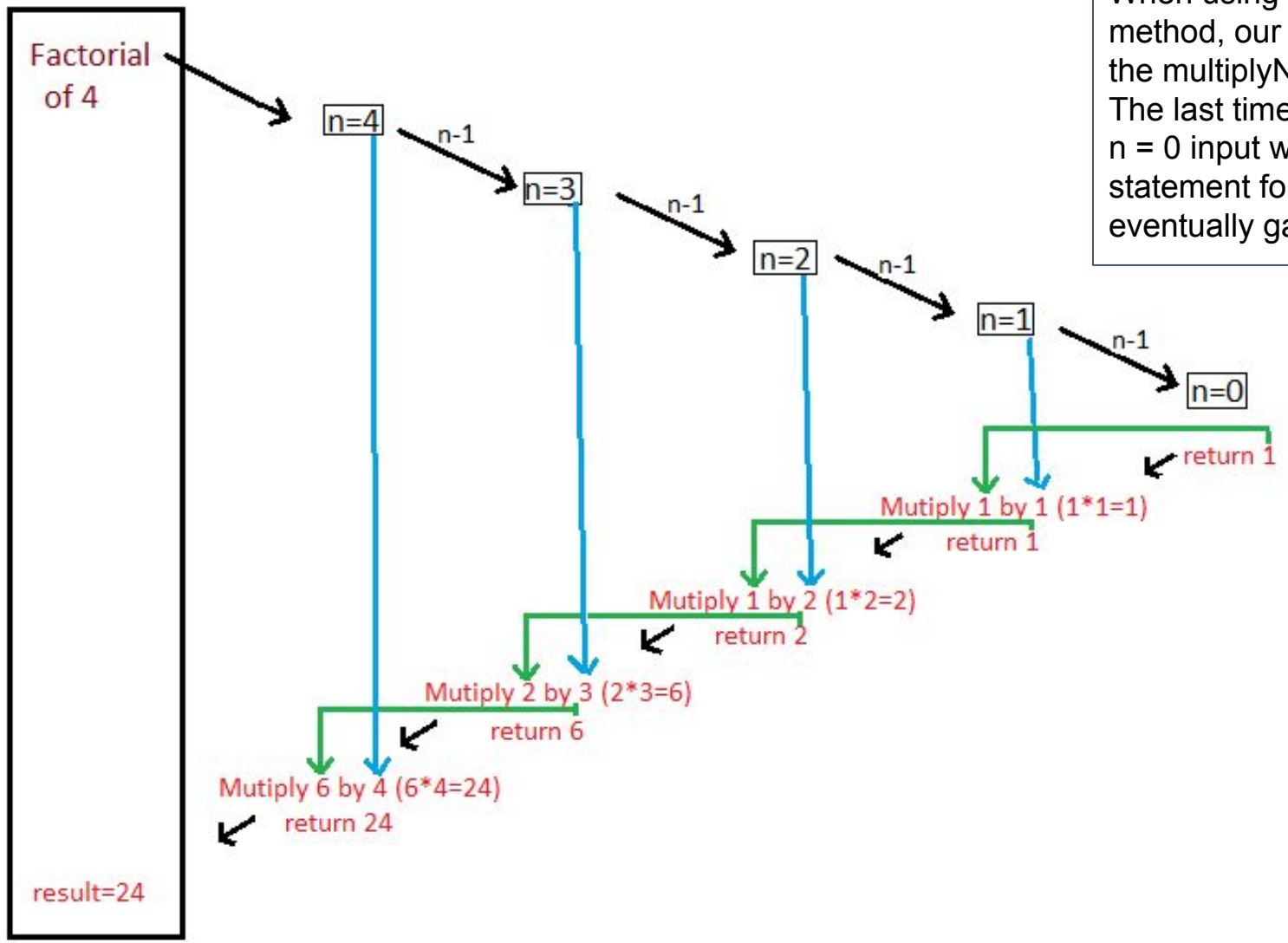
```java
public class FirstProject {
    public static long multiplyNumbers(int num) {
        if (num >= 1) {
            return num * multiplyNumbers(num - 1);
        } else {
            return 1;
        }
    }

    public static void main(String[] args) {
        int num = 4;
        long factorial = multiplyNumbers(num);
        System.out.println("Factorial of " + num + " = " + factorial);
    }
}
```

The multiplyNumber method is calling itself, meaning that this method is a recursive method

```
Factorial of 4 = 24
```

Java Recursion Example Explained



When using the recursive factorial method, our Java compiler triggered the multiplyNumber method 5 times. The last time it trigger the method with n = 0 input which execute the return statement for all of them and eventually gave the output of 24

Class Exercise - Java Recursion

Instructions:

- On your existing Java project create a new Java file and call it “JavaRecursion”.
- Create a main() method and write a recursion method that gets a range of numbers and return the multiplication of all the numbers in this range.
- For example:
  - For the input (4, 6) the function output should be → 120 (4*5*6)
  - For the input (2,7) the function output should be → 5040 (2*3*4*5*6*7)

Class Exercise Solution - Java Recursion



Java Exception Handling

Java Exceptions

When working with Java we can see 2 different types of errors:

- **Compilation error -** An error that occur during the compilation process.
  We can fix this error before our users see it by themselves.
- **Runtime error -** An error that occur during our application runtime.
  Only by running our application we can notice this kind or error.

Note: When we see compilation errors, we couldn’t run our Java application until we fix those errors. Unlike runtime errors, which we could not understand that we have them until we or our users use our Java application and it throw those errors.

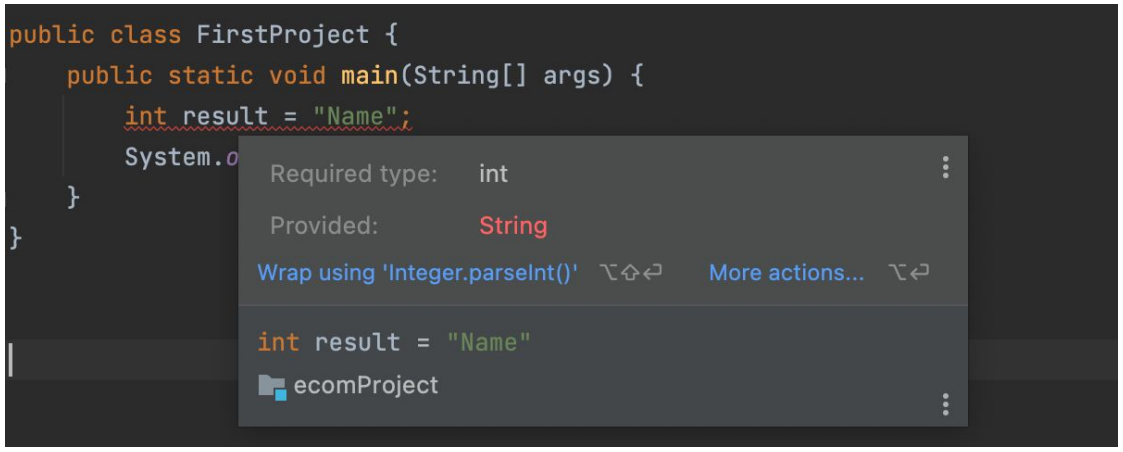
Java Exceptions Example

Runtime error example:

```java
public class FirstProject {
    public static void main(String[] args) {
        int[] array = {1, 2, 3, 4, 5, 6};
        System.out.println(array[6]);
    }
}
```

```
Index 6 out of bounds for length 6
```

Compilation error example:



Error Handling (Try & Catch)

When an error occurs, Java will normally stop and generate an error message. The technical term for this is: Java will **throw an exception** (throw an error).

We can catch exceptions and handle them with the **try {}** and **catch ()** keywords.

try {} - The **try** statement allows you to define a block of code to be tested for errors while it is being executed.

catch() - The **catch** statement allows you to define a block of code to be executed, if an error occurs in the try block.

```
try {
      //  Block of code to try
} catch(Exception e) {
      //  Block of code to handle errors
}
```

Error Handling (Try & Catch) Example

```java
public class FirstProject {
    public static void main(String[ ] args) {
        try {
            int[] myNumbers = {1, 2, 3};
            System.out.println(myNumbers[10]);
        } catch (Exception e) {
            System.out.println("Something went wrong.");
        }
    }
}
```

The try{} block handle the NullPointerException

```
Something went wrong.
```

The catch() block catch the NullPointerException and print “Something went wrong.” instead of failing the program

The finally Keyword

The **finally** statement lets you execute code, after **try...catch**, regardless of the result.

Meaning that the block inside the finally statement will be always executed.

```
try {
    //  Block of code to try
} catch(Exception e) {
    //  Block of code to handle errors
} finally {
    // Block of code that always be executed
}
```

}



ECOM SCHOOL

המכללה למקצועות הדיגיטל וההייטק

The finally Keyword Example

```java
public class FirstProject {
    public static void main(String[] args) {
        try {
            int[] myNumbers = {1, 2, 3};
            System.out.println(myNumbers[10]);
        } catch (Exception e) {
            System.out.println("Something went wrong.");
        } finally {
            System.out.println("The 'try catch' is finished.");
        }
    }
}
```

```
Something went wrong.
The 'try catch' is finished.
```

When executing this program, the code inside the finally {} block will always be executed, regardless the try..catch result



ECOM SCHOOL

המכללה למקצועות הדיגיטל וההייטק

Error Handling - Throw Custom Exceptions

In cases we want to create **custom errors** we can throw our own exception anywhere we want in the code by using the **throw** Java keyword.

The **throw** statement is used together with an exception type. There are many exception types available in Java: ArithmeticException, FileNotFoundException, ArrayIndexOutOfBoundsException, SecurityException, etc…

In addition, when we want to create a new Java exception to throw we need to use the **new** Java keyword.
We will learn more about the **new** keyword later in the course.



Throw Custom Exceptions Example

```java
public class FirstProject {
    static void checkAge(int age) {
        if (age < 18) {
            throw new ArithmeticException("Access denied - You must be at least 18 years old.");
        }
        else {
            System.out.println("Access granted - You are old enough!");
        }
    }

    public static void main(String[] args) {
        checkAge(15); // Set age to 15 (which is below 18...)
    }
}
```

```
Access denied - You must be at least 18 years old.
```

When the if() block will be executed, our program will throw new custom exception (ArithmeticException)

Class Exercise - Java Error Handling

Instructions:

- On your existing Java project create a new Java file and call it “JavaExceptions”.
- Create a main() method.
- Your job is to create a program that gets candidate name and candidate job title.
- Write a function that check if the job title is not 1 of the above: Fullstack, Backend, Frontend.
- If no → your program should throw new IlligalArgumentException with the message: “The provided job title is unsupported”
- If yes → your function should return the corresponding salary of that job.
    - For Fullstack → 22,000
    - For Backend → 20,000
    - For Frontend → 18,000

Class Exercise - Java Error Handling

Instructions:

- Write another program that takes the corresponding job salary and calculate how much money this candidate should earn in 5 years working at this job.
- When finish, your function should print this sentence: “Hello {candidate name}, with the job title {candidate job title} you will earn in 5 years: {calculated money} shekels”
- Your program should know how to handle errors so it won’t fail regardless the output of the user.
- In case of an error, your function should print: “Hello {candidate name}, the job title you provide {candidate job title} is unknown to us, so we can’t calculate your salary in 5 years”

Class Exercise Solution - Java Error Handling